Report

Emanuel Paraschiv

March 2025

1 Introduction

The scope of this report is to present my idea and implementation for Task 2 in the StreamCipher assignment.

2 Linear Congruential Generator

$X_(i+1)$=(x_i*a + b) mod m

Among the goals of designing a great random generator, we must ensure that the period is large, namely that the sequence eventually repeats after a long enough time.

Another important factor is to ensure that there are no noticeable patters in the sequence output, otherwise it would be easy to 'guess' what is to come next. Among other tests, to ensure this I plotted my results on a bitmap to ensure random distribution, and the results are visible below:

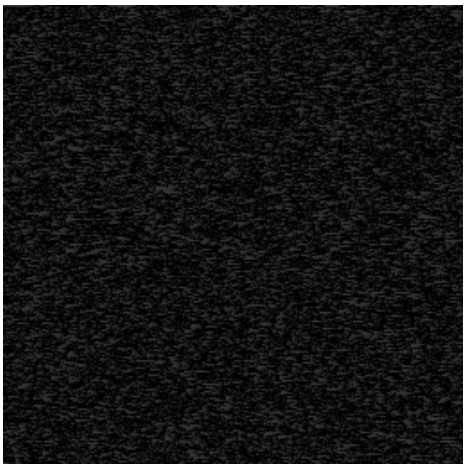

Figure 1: Bitmap plot

In order to choose m, we must ensure that it is a large prime number. In order to do that I selected it to be a large power of 2, and used Mathematica software to find it:

```
m = NextPrime[2^30];
```

This operation yielded 1073741827 as a result.This large value will ensure a big period along with a more even distribution of values.

In my logic I have kept b to be 0, and this way we have a multiplicative LCG. If we were to choose an increment number, this should come with the property of being coprime to m.

As a must be a primitive root of m, I used the **PrimitiveRoot** function:

```
a = PrimitiveRoot[m];
```

This again yielded 2 as a result. Even though other primitive roots get a more clear and lighter image, this first primitive root fulfills our scope for the moment.

The psedo-random number generator implemented using the next() method will take the previously generator number as the seed, and will generate the next number in the sequence using LCG. Then, the method will only return the number of bits specified by the user. This works by using a mask of 1's that is &'ed with the resulting number.